Management of Telecom Projects

Task 11: Unicode File Transfer

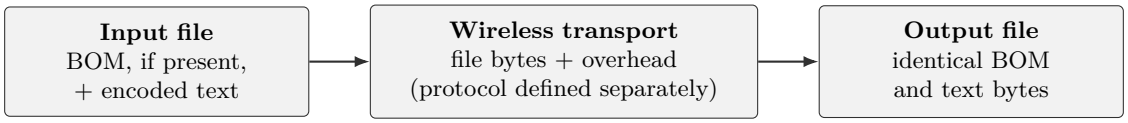
Guillem Moreno
Team C

October 2026

1. Input format and transfer method

The competition supplies Unicode text files, but does not specify their encoding, byte order or use of a byte order mark. The transfer method must therefore preserve files encoded in UTF-8, UTF-16 or UTF-32 without requiring their interpretation.

The file is read in binary mode into a `uint8_t` buffer. Each read returns n bytes, which are passed unchanged to the transport. A byte is an 8-bit value from 0 to 255. Buffer lengths are stored explicitly: `strlen()` is unsuitable because `00` may occur within the file. No hexadecimal or Base64 conversion is required.



2. Byte order mark

A plain `.txt` file has no mandatory universal header. It may begin with a **byte order mark (BOM)**, the encoded value U+FEFF used as an encoding signature. In UTF-16 and UTF-32, the BOM also indicates byte order. When interpreted as a BOM, it is not part of the displayed text.

Encoding	Initial BOM bytes	Meaning for a text decoder
UTF-8	`EF BB BF`	Encoding signature only. UTF-8 has no little-/big-endian variants.
UTF-16 LE	`FF FE`	Read 16-bit units, least-significant byte first.
UTF-16 BE	`FE FF`	Read 16-bit units, most-significant byte first.
UTF-32 LE	`FF FE 00 00`	Read 32-bit units, least-significant byte first.
UTF-32 BE	`00 00 FE FF`	Read 32-bit units, most-significant byte first.

For example, A (U+0041) is stored as `41` in UTF-8, `41 00` / `00 41` in UTF-16 LE/BE, and `41 00 00 00` / `00 00 00 41` in UTF-32 LE/BE.

3. Encoding identification

For an unlabelled text file, the initial BOM normally identifies the encoding. Four-byte signatures are checked before two-byte signatures, followed by strict validation of the encoded text. Transfer and storage preserve the BOM in its original position, regardless of the decoding result.

The absence of a BOM does not identify UTF-8. UTF-16 and UTF-32 files may also omit it. For instance, `41 00` represents A followed by NUL in UTF-8, or A in UTF-16 LE. Selecting UTF-8 by default requires an explicit agreement about the input format.

Some signatures overlap. The bytes `FF FE 00 00 41 00 00 00` represent either a UTF-32 LE BOM followed by A, or a UTF-16 LE BOM followed by NUL, A and NUL. Checking the longest signature first selects UTF-32 LE, but both interpretations are valid. A declared encoding or an agreed file convention resolves this case.

Character and line processing require a known encoding. A separate copy of the file is decoded with strict error handling, consuming only the initial BOM when applicable. The saved original retains all bytes, including the BOM and line endings. Missing or conflicting encoding information must be resolved before text processing.

4. File input and output

The source is opened in binary mode (`rb`). The receiver writes the recovered file bytes in their original order, excluding transport headers and padding. The filename and filesystem metadata are stored separately from file content. The transport header format is outside the scope of this report.

The following Python example writes recovered data on a PC. The sequence `ordered_file_chunks` contains only file bytes. Mode `xb` creates a new binary file and fails if the destination already exists.

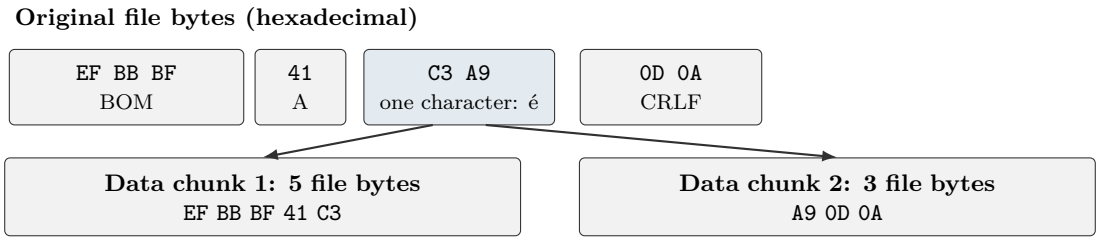
```
with open(new_path, "xb") as output:
    for chunk in ordered_file_chunks:
        if output.write(chunk) != len(chunk):
            raise OSError("Incomplete file write")
```

The embedded implementation needs equivalent binary filesystem operations and error checks for reading, writing and closing. Buffered data must be flushed and the USB safely unmounted before removal. The saved byte count and contents are checked against the source. Text processing operates on a decoded copy, leaving the received file unchanged.

5. File reading and chunk boundaries

File access uses the operating system's filesystem API, or the filesystem library on an embedded platform. Binary reads return bytes without selecting a text encoding. For text processing, the application checks the first four file bytes, or fewer for a shorter file, against the BOM signatures and selects the corresponding decoder. An absent or ambiguous BOM requires a declared encoding or an agreed input format. Decoding does not alter the bytes transferred or saved.

A binary read may split a multibyte character. The text `Aé` followed by a CRLF line ending occupies eight bytes in UTF-8 with a BOM. Reading five bytes and then three divides the two-byte representation of é across the reads.



Reassembly restores the original bytes, including the split character.

Text is decoded after reassembly or with an incremental decoder that retains incomplete sequences between reads. Independent decoding of each chunk can fail when a character or BOM crosses a boundary.

6. File size

File size includes the encoded text, line endings and any BOM. Transport overhead is additional. Copying the bytes without conversion preserves the original file size.

For 1,000 ASCII letters with a BOM and no line breaks, the file sizes are:

Encoding	Text bytes	BOM bytes	Total file bytes
UTF-8	1,000	3	1,003
UTF-16 LE / BE	2,000	2	2,002
UTF-32 LE / BE	4,000	4	4,004

Omitting the BOM gives sizes of 1,000, 2,000 and 4,000 bytes. Byte order does not affect size. The ratios shown apply to ASCII; other character sets produce different byte counts.